

Управление на книги в библиотека-упражнение

Създайте уеб приложение, което позволява да се управляват база данни с книги.

Функционалности (CRUD операции):

- ✓ **Създаване (Create)** – Добавяне на нова книга с информация като заглавие, автор, година на издаване, жанр описание.
- ✓ **Четене (Read)** – Преглед на списък с наличните книги и детайлна страница за всяка книга.
- ✓ **Редактиране (Update)** – Възможност за редактиране на информацията за дадена книга.
- ✓ **Изтриване (Delete)** – Премахване на книга от списъка.

! След всяка нова стъпка натискаме клавишна комбинация Ctrl + S, за да се запази, съответно разпознае и да може да бъде използвана в следващата стъпка.

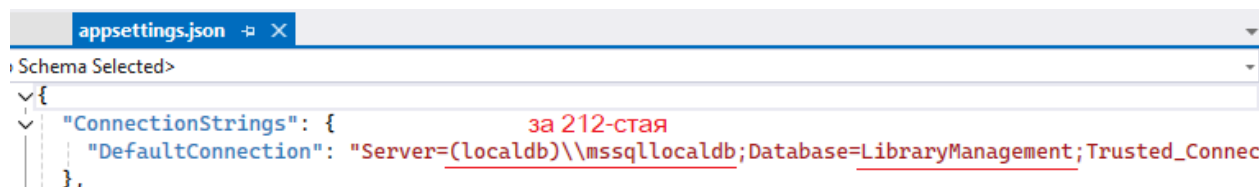
Ръководство за създаване на уеб приложение "Управление на библиотека" с ASP.NET MVC

1. Създаване на ASP.NET MVC проект

Отваряне на **Visual Studio** и създаване на нов проект:

- Избиране на **ASP.NET Core Web App (Model-View-Controller)**.
- Избиране на **.NET 5.0**.
- Задаване на име, например `LibraryManagement`.
- Избиране на **Individual Accounts** за аутентикация (ако се използва вход за потребители).

Задайте настройка за сървър на стаята и подходящо име за БД с вашия клас и паралелка.



2. Добавяне на моделите (Models)

Клас за книгите (Book.cs)

```
Book.cs
using System.ComponentModel.DataAnnotations;

namespace LibraryManagement.Models
{
    public class Book
    {
        public int Id { get; set; }

        [Required]
        [StringLength(100)]
        public string Title { get; set; }

        [Required]
        [StringLength(50)]
        public string Author { get; set; }

        public string Genre { get; set; }

        public int Year { get; set; }

        public byte[] Cover { get; set; }

        public bool IsAvailable { get; set; } = true;
    }
}
```

```
public int Id { get; set; }

[Required]
[StringLength(100)]
public string Title { get; set; }

[Required]
[StringLength(50)]
public string Author { get; set; }

public string Genre { get; set; }

public int Year { get; set; }

public byte[] Cover { get; set; }

public bool IsAvailable { get; set; } = true;
```

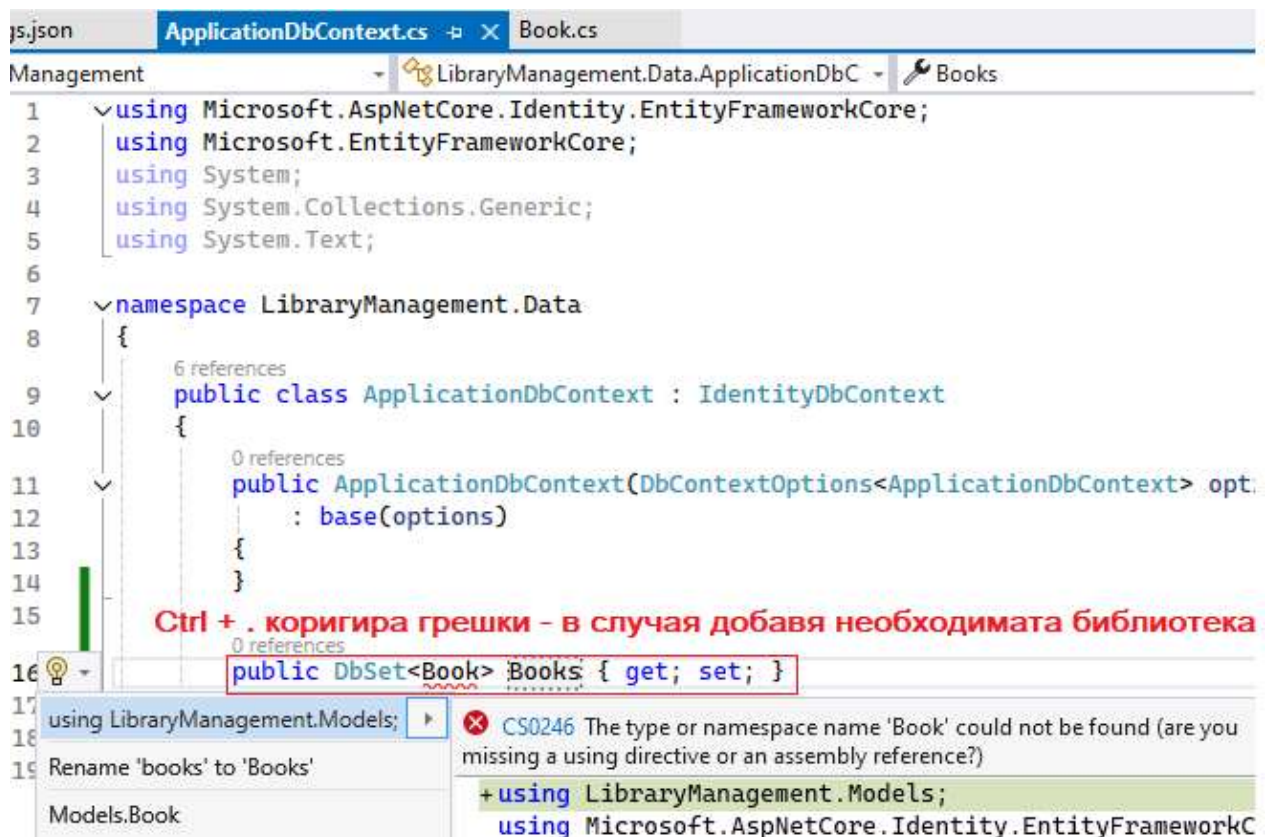
3. Конфигуриране на база данни (Entity Framework Core)

Добавяне на DbSet<Book> в ApplicationDbContext

Принципно, досега при скафолдове (генериране на автокод) създавахме празен конструктор и преминавахме към създаване на контролера с 5-те изгледа. При този начин, не навлизахме в ApplicationDbContext, а само го извиквахме за контролера. Сега, обаче, т.к. ще използваме картинка, този начин на представяне на информация няма да бъде достатъчен за нейното изобразяване и редактиране. Затова ще се наложи първо да редактираме AppDbContext, за да може картинката да бъде добавена в БД.

Добавяме само един необходим ред:

```
public DbSet<Book> Books { get; set; }
```



Изпълнение на миграции за създаване на БД:

В Package Manager Console:

```
Add-Migration InitialCreate  
Update-Database
```

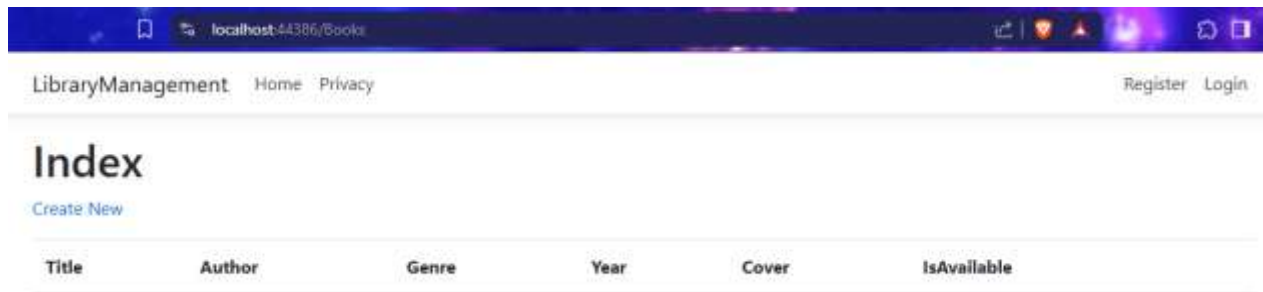
4. Автоматично генериране на CRUD контролер и изгледи

Във **Visual Studio**:

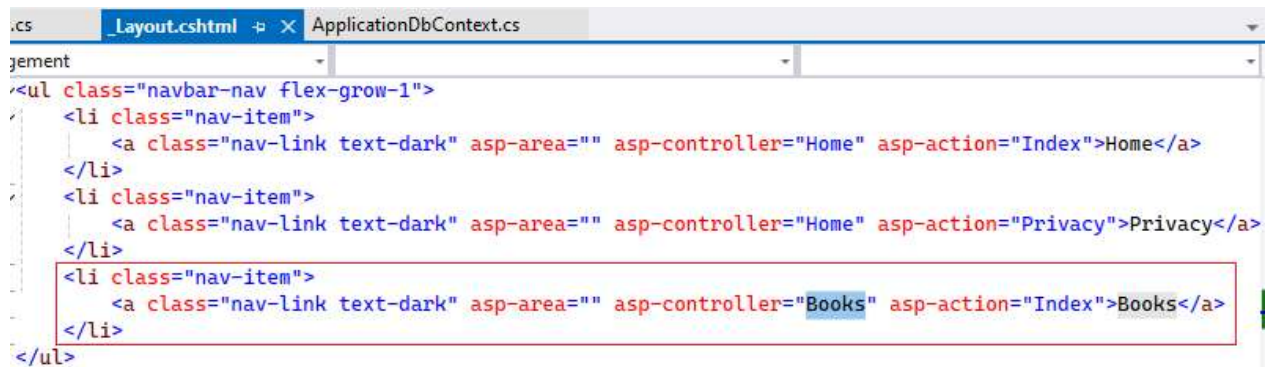
1. В **Solution Explorer** с десен бутон върху папката `Controllers`.
2. **Add** → **Controller**.
3. **MVC Controller with views, using Entity Framework**.
4. Избира се **Book** като модел и **ApplicationDbContext** като контекст на базата данни.
5. **Add**.

5. Потребителски интерфейс в изгледите на папка `Views/Books`.

Можем да стартираме проекта и разгледаме резултата, като допишем в URL-адреса името на контролера `Books`



През URL-адреса винаги може да се навигира до съответната страница, действие и id, независимо, че те не са приложени все още интерактивно, така че да бъдат видими за потребителите през `_Layout.cs`. Това може да бъде направено сега:



Create

Book

Title

Author

Genre

Year

Cover

☐ IsAvailable

В страницата Create забелязваме, че няма възможност за прикачване на файл с изображение, затова добавяме тази функционалност – на първо място задаваме криптиране на самата форма, за да може да използва не само низ от текст/цифри, но и по-сложен матричен като изображение. После работим върху самия бутон за избор на файл - изтриваме валидацията, която няма да ни бъде необходима, задаваме тип на input – с възможност за прикачване на файл и статично име ImageFile, което ще бъде използвано като нови входни данни в метод Create на контролера.

```

Book.cs  Create.cshtml  BooksController.cs
nagement
<div class="col-md-4">
  <form asp-action="Create" enctype="multipart/form-data">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>
    <div class="form-group">...
    <div class="form-group">...
    <div class="form-group">... СВИТИ
    <div class="form-group">...
    <div class="form-group" asp-action="Create">
      <label asp-for="Cover" class="control-label"></label>
      <input type="file" name="ImageFile" class="form-control" />
      изтрива <span asp-validation-for="Cover" class="text-danger"></span>
    </div>
  </form>

```

```

<form asp-action="Create" enctype="multipart/form-data">
  .....
  <div class="form-group" asp-action="Create">
    <label asp-for="Cover" class="control-label"></label>
    <input type="file" name="ImageFile" class="form-control" />
  </div>

```

6. В методите Create и Edit на BooksController добавяме логика за качване на изображение:

Нови понятия свързани с обработката на изображения в ASP.NET Core

- **IFormFile** е интерфейс в ASP.NET Core, който се използва за качване на файлове чрез формуляри в уеб приложения.
- **MemoryStream** е обект в C#, който позволява временно съхраняване на данни в паметта (RAM), вместо в файл или база данни. **Каченото** изображение първо се копира в MemoryStream, /преобразува изображение в масив от байтове/ а след това го записва в БД
- В **BooksController** използваме **IFormFile imageFile**, за да прихванем качения файл от формуляра. Независимо, че в самия изглед се използва по конвенция главна буква ImageFile, а в контролера се подава за разпознаване променлива с малка буква imageFile, самият алгоритъм се осъществява благодарение на разпознаването на името на самия action – Create и неговият метод Post за изпращане. По конвенция, интерфейсите винаги започват с главна буква I.

```

[HttpPost]
[ValidateAntiForgeryToken]
public async Task<ActionResult> Create([Bind("Id,Title,Author,Genre,Year,Cover,IsAvailable")] Book book, IFormFile imageFile)
{
    if (ModelState.IsValid)
    {
        if (imageFile != null && imageFile.Length > 0)
        {
            using (var memoryStream = new MemoryStream())
            {
                await imageFile.CopyToAsync(memoryStream);
                book.Cover = memoryStream.ToArray();
            }
        }
        _context.Add(book);
        await _context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }
    return View(book);
}

```

Проверка дали е качен файл

Създаване на MemoryStream, за да се запази в паметта
Копиране на файла в MemoryStream
Преобразуване на файла в масив от байтове byte[] и
записване в book.Cover

Запазване на книгата в БД

```

if (imageFile != null && imageFile.Length > 0)
{
    using (var memoryStream = new MemoryStream())
    {
        await imageFile.CopyToAsync(memoryStream);
        book.Cover = memoryStream.ToArray();
    }
}

```

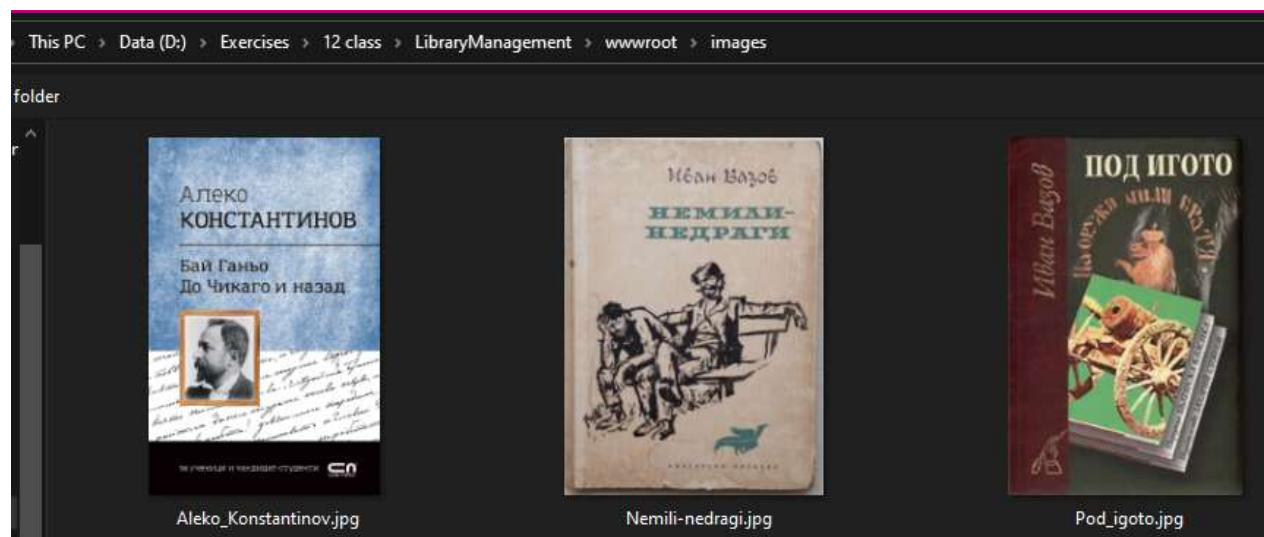
Cover

Choose File No file chosen

☐ IsAvailable

Create

При тест, вече бутонът за избор на файл работи. Тук е моментът да създадем нова папка images в wwwroot, в която да запазим няколко корици:



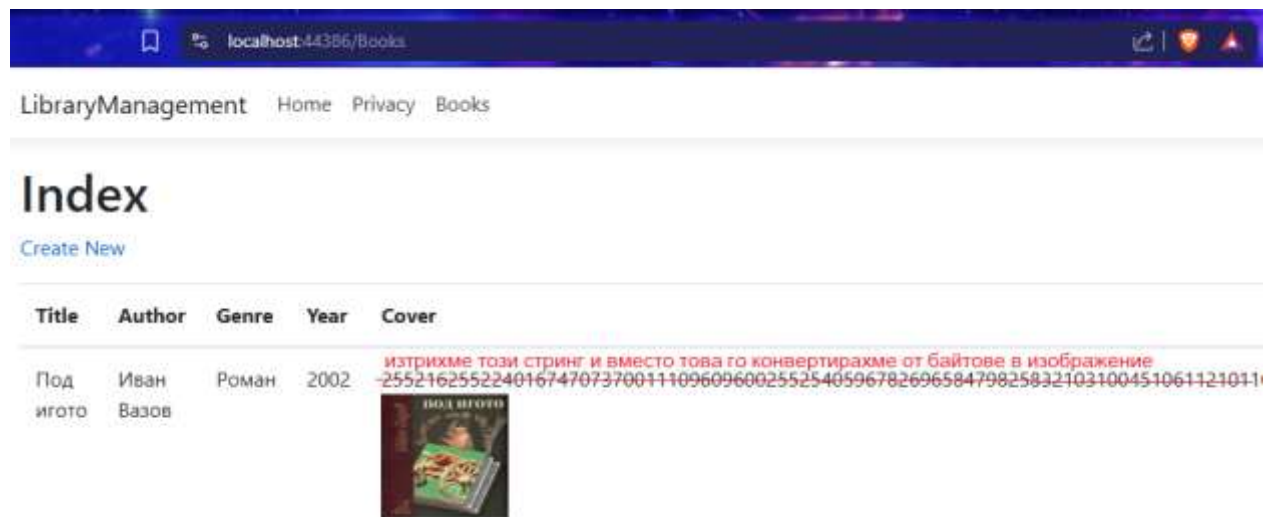
Макар при проверка, картинките да са запазени в БД, те все още няма да се изобразят в index.cshtml, докато не променим страницата.

```
ate.cshtml | Index.cshtml* | BooksController.cs
ent
<td>
    @Html.DisplayFor(modelItem => item.Cover) изтриваме
    @if (item.Cover != null && item.Cover.Length > 0)
    {
        
    }
    else
    {
        <span>Няма изображение</span>
    }
</td>
```

```
@if (item.Cover != null && item.Cover.Length > 0)
{
    
}
else
{
    <span>Няма изображение</span>
}
```

item.Cover е byte[], който съдържа двоичните данни за изображението

img src="data:image/png;base64 казва на брауъра, че това е изображение в base64 формат
Convert.ToBase64String(item.Cover) преобразува масива от байтове в base64 низ



По същия начин правим и Edit – той повтаря кода на Create:

```

te.cshtml Edit.cshtml Book.cs Create.cshtml Index.cshtml B
<form asp-action="Edit" enctype="multipart/form-data">
  <div asp-validation-summary="ModelOnly" class="text-danger"></div>
  <input type="hidden" asp-for="Id" />
  <div class="form-group">...
  <div class="form-group">...
  <div class="form-group">...
  <div class="form-group">...
  <div class="form-group">
    <label asp-for="Cover" class="control-label"></label>
    <input type="file" name="ImageFile" class="form-control" />
  </div>
</form>

```

```

troller.cs Delete.cshtml* Details.cshtml
<dd class = "col-sm-10">
  @Html.DisplayFor(model => model.Cover) изтриваме, но виждаме model.
  @if (Model.Cover != null && Model.Cover.Length > 0) item. замества с Model.
  {
    
  }
  else
  {
    <span>Няма изображение</span>
  }
</dd>

```

```

troller.cs Delete.cshtml
<dd class = "col-sm-10">
  @Html.DisplayFor(model => model.Cover)
  @if (Model.Cover != null && Model.Cover.Length > 0)
  {
    
  }
  else
  {
    <span>Няма изображение</span>
  }
</dd>

```

```
BooksController.cs - X
LibraryManagement.Controllers.BooksController Create(Book book, IFormFile imageFile)

[HttpPost]
[ValidateAntiForgeryToken]
0 references
public async Task<IActionResult> Edit(int id, [Bind("Id,Title,Author,Genre,Year,Cover,IsAvailable")] Book book)
{
    if (id != book.Id)
    {
        return NotFound();
    }

    if (ModelState.IsValid)
    {
        try
        {
            _context.Update(book);
            await _context.SaveChangesAsync();
        }
        catch (DbUpdateConcurrencyException)
        {
            if (!BookExists(book.Id))
            {
                return NotFound();
            }
            else
            {
                throw;
            }
        }
        return RedirectToAction(nameof(Index));
    }
}
```

ще променим кода

```
ex.cshtml BooksController.cs - X
LibraryManagement.Controllers.BooksController Edit(int id, Book book, IFormFile imageFile)

[HttpPost]
[ValidateAntiForgeryToken]
0 references
public async Task<IActionResult> Edit(int id, [Bind("Id,Title,Author,Genre,Year,Cover,IsAvailable")] Book book, IFormFile imageFile)
{
    if (id != book.Id)
    {
        return NotFound();
    }

    if (imageFile != null && imageFile.Length > 0) // Проверяваме дали има качен нов файл
    {
        using (var memoryStream = new MemoryStream())
        {
            await imageFile.CopyToAsync(memoryStream);
            book.Cover = memoryStream.ToArray(); // Записваме новото изображение
        }
    }
    else // Ако потребителят не качи ново изображение, запазваме старото
    {
        var existingBook = await _context.Books.FindAsync(id);
        if (existingBook != null)
        {
            book.Cover = existingBook.Cover;
        }
    }
    _context.Update(book);
    await _context.SaveChangesAsync();
    return RedirectToAction(nameof(Index));
}
```

като в Create

```
if (imageFile != null && imageFile.Length > 0) // Проверяваме дали има качен нов
файл
{
    using (var memoryStream = new MemoryStream())
    {
        await imageFile.CopyToAsync(memoryStream);
        book.Cover = memoryStream.ToArray(); // Записваме новото изображение
    }
}
```

```

    }
}
else // Ако потребителят не качи ново изображение, запазваме старото
{
    var existingBook = await _context.Books.FindAsync(id);
    if (existingBook != null)
    {
        book.Cover = existingBook.Cover;
    }
}
_context.Update(book);
await _context.SaveChangesAsync();
return RedirectToAction(nameof(Index));

```

Страница Edit може да се подобри още:

```

<cs Edit.cshhtml Index.cshhtml BooksController.cs
<form asp-action="Edit" enctype="multipart/form-data">
  <div asp-validation-summary="ModelOnly" class="text-danger"></div>
  <input type="hidden" asp-for="Id" />
  <div class="form-group">...
  <div class="form-group">...
  <div class="form-group">...
  <div class="form-group">...
  <div class="form-group">
    <label asp-for="Cover" class="control-label"></label>
    <br />
    @if (Model.Cover != null && Model.Cover.Length > 0)
    {
        
    }
    else
    {
        <span>Няма изображение</span>
    }
    <br />
    <input type="file" name="ImageFile" class="form-control" />
  </div>
</form>

```

Резултат:

Title

Под игото

Author

Иван Вазов

Genre

Роман

Year

2002

Cover



Choose File No file chosen

☒ IsAvailable

Save

[Back to List](#)

Благодарение на вложената проверка за валидност в Create – на модела и на самото изображение, се допуска създаване на книга без корица:

Index

[Create New](#)

Title	Author	Genre	Year	Cover	IsAvailable	
Под игото	Иван Вазов	Роман	2002		☒	Edit Details Delete
Немили-недраги	Иван Вазов	Повест	1965		☐	Edit Details Delete
До Чикаго и назад, Бай Ганьо	Алеко Константинов	Пътепис, фейлетон	2022	Няма изображение	☒	Edit Details Delete

А също и добавяне на корица впоследствие при редактиране:

Index

[Create New](#)

Title	Author	Genre	Year	Cover	IsAvailable	
Под игото	Иван Вазов	Роман	2002		☒	Edit Details Delete
Немили-недраги	Иван Вазов	Повест	1965		☐	Edit Details Delete
До Чикаго и назад, Бай Ганьо	Алеко Константинов	Пътепис, фейлетон	2022		☒	Edit Details Delete

Има и бърз начин на превод на колоните на всички 5 изгледа от Views/Books едновременно, вместо страничка по страничка – ако директно им зададем име в модела:

```

10 references
public int Id { get; set; }

[Required]
[StringLength(100)]
[Display(Name="Заглавие")]
12 references
public string Title { get; set; }

[Required]
[StringLength(50)]
[Display(Name = "Автор")]
12 references
public string Author { get; set; }

[Display(Name = "Жанр")]
12 references
public string Genre { get; set; }

[Display(Name = "Година")]
12 references
public int Year { get; set; }

[Display(Name = "Корица")]
21 references
public byte[] Cover { get; set; }

[Display(Name = "Налична")]
10 references
public bool IsAvailable { get; set; } = true;

```

Заглавие	Автор	Жанр	Година	Корица	Налична	
Под игото	Иван Вазов	Роман	2002		☒	Edit Details Delete
Немили-недраги	Иван Вазов	Повест	1965		☐	Edit Details Delete
До Чикаго и назад, Бай Ганьо	Алеко Константинов	Пътепис, фейлетон	2022		☒	Edit Details Delete

Заглавие

Под игото

Автор

Иван Вазов

Жанр

Роман

Година

2002

Корица



Choose File

No file chosen



Налична

Save

Може бутоните да се стилизират:

Заглавие	Автор	Жанр	Година	Корица	Налична	
Под игото	Иван Вазов	Роман	2002		☐	Edit Details Delete
Немили-недраги	Иван Вазов	Повест	1965		☐	Edit Details Delete
До Чикаго и назад, Бай Ганьо	Алеко Константинов	Пътепис, фейлетон	2022		☐	Edit Details Delete

Това става чрез добавяне на клас в изгледа:

```

site.css Delete.cshtml Edit.cshtml Index.cshtml BooksController.cs
agement
v<td>
    <a asp-action="Edit" asp-route-id="@item.Id" class="btn-primary">Edit</a> |
    <a asp-action="Details" asp-route-id="@item.Id" class="btn-info">Details</a>
    <a asp-action="Delete" asp-route-id="@item.Id" class="btn-danger">Delete</a>
</td>

```

А заоблянето на 3-те класа едновременно става глобално в site.css:

```

site.css Delete.cshtml Edit.cshtml
.btn-primary, .btn-info, .btn-danger {
    border-radius: 50px;
    padding: 5px 10px;
}

.btn-primary, .btn-info, .btn-danger {
    border-radius: 50px;
    padding: 5px 10px;
}

```